SAP ABAP Development Standards

Document Version History

Date	Version	Description	Author
11-03-2020	0.1	Overview of Coding Standards and Development Best Practices	Veda
26-07-2021	0.2	ABAP on HANA - Additions	Veda

1. Objective

The purpose of this document is to deliver a consistent set of conventions and guidelines for developing SAP applications with the goal of maximizing the quality, value and maintainability of the developed solution. It is essential that every person involved in the development process familiarizes him/herself with this document and applies the described standards and processes.

When maintaining existing code, it is expected that these standards be retrospectively applied.

Please Note – Customer Naming Conventions take precedence over this, if exists.

2. Naming conventions

The following sections explain the naming conventions for all ABAP repository objects. Naming conventions of objects not mentioned here should be decided within the development team.

2.1 Object Naming Conventions

XX – Module Code (See Appendix A for list of module codes)

Object	Format	Example(s)
Classes	ZCL_XX_<Description>	ZCL_FI_CUSTOMER_CREATE
Interfaces	ZIF_XX_<Description>	ZIF_FI_CUSTOMER_CREATE
Tables	ZXX_<Description>	ZMM_PROD_HIER
Field Names in Custom tables	Z<Description> or a standard SAP name if being used for the same purpose as WAERS for currency.	ZCUSTOMER_NAME or WAERS
Data Elements	ZXX_<Description>	ZMM_PROD_HIER
Domains	ZXX_<Description>	ZMM_PROD_HIER
Function Groups	ZFGXX_<Function Group>	ZFGSD_PRICING_MASTER
BADI	ZXX_<BADI_DEFINITION>	ZFI_CUSTOMER_UPD
Projects (Customer Enhancements)	ZXX_<Description>	ZFI_CUSTOMER

Enhancement Spot	ZXX_ES_<Enhancement Spot Name>	ZFI_ES_CUSTOMER_UPD
Composite Enhancement Spot	ZXX_CS_< Composite Enhancement Spot Name >	ZFI_CS_CUSTOMER_UPD
Enhancement Point	ZXX_EP_< Enhancement Point Name >	ZFI_EP_CUSTOMER_UPD
Enhancement Implementations	ZXX_EI_< Enhancement Implementation Name >	ZFI_EI_CUSTOMER_UPD
Composite Enhancement Implementations	ZXX_CI_< Composite Enhancement Implementation Name >	ZFI_CI_CUSTOMER_UPD
Function Modules	ZXX_<FunctionModuleName>	ZFI_CUSTOMER_ADDR_UPD
Programs	ZXX_<Description>	ZFI_MASS_ASSET_TRANSFER
Includes	ZIXX_<IncludeName>	ZIFI_MASS_ASSET_TRANSFER_TOP
Message Class	ZXX_<Description>	ZFI_MSG
Search Helps	ZXX_<Description>	ZFI_GET_CUST_BANK_DETAILS
Smart forms	ZXX_<Description>	ZFI_INVOICE_PRINT
Structures	ZSXX_<Description>	ZSFI_VENDOR_LIST
Table Types	ZTTXX_<Description>	ZTTFI_VENDOR_LIST
Lock Object name	EZ_<Tablename>	EZ_KNA1
Transactions	ZXX_<Description>	ZFI_ASSET_POST
Views	ZYXX_<Description>	ZVMM_PROD_HIER
Odata Project Name	ZXX_<Description>	ZFI_POST_CUSTOMER
Web Service	ZWS_<Description>	ZWS_PRODUCT_MASTER
Workflow Standard Task	ZTSXX_<RIVEFNO>_<Description>	ZTSFI_INVOICE_APPR
Workflow Task Group	ZTGXX_<Description>	ZTGFI_INVOICE_APPR

Workflow Rules	ZRUXXX_<Description>	ZRUFI_INV

2.2 Code Conventions

Object	Format	Example(s)	Notes
Field Symbols	<FS_<Description>	<FS_INPUT>	Field Symbols should be typed wherever possible (use type 'DATA' or type 'ANY')
Global Constants Local Constants	GC_<Description> LC_<Description>	GC_TRUE LC_TRUE	Constants should generally be defined as global constants. Use LC_ if local constants are required
Types	TY_<Description>	TY_ORDER_DATA	
Global Variables	GV_<Description> GT_<Description> GS_<Description> GR_<Description>	GV_AUFNR GT_ORDER_DATA GS_ORDER_DATA GR_ALV	GV -> Variables GT -> Internal Tables GS -> Structures GR -> Ranges
Local Variables	LV_<Description> LT_<Description> LS_<Description> LR_<Description>	LV_AUFNR LT_ORDER_DATA LS_ORDER_DATA LR_ALV	LV -> Variables LT -> Internal Tables LS -> Structures LR -> Ranges
Parameters*	IV_<Description> EX_<Description> CH_<Description> IT_<Description> ET_<Description> CT_<Description> R_<Description>	IM_ORDER EX_ORDER CH_ORDER IT_ORDERS ET_ORDERS CT_ORDERS R_KUNNR	IM -> Import EX -> Export CH -> Changing IT -> Import Tables ET -> Export Tables CT -> Changing Tables R-> Returning parameters from methods

Selection Screen Parameters	S_<Description> P_<Description>	S_LIFNR P_DATE	Use S for Select-Options Use P for single selection screen parameters

*Note Parameter naming conventions apply to Forms, Function Modules, and Class Method Interfaces.

3. ABAP Programming Guidelines

Programming is an individually controlled process and different developers have different styles and different preferences. However, to create quality development a basic set of rules is necessary to standardize development that is flexible enough to meet possible future requirements.

- Simplicity

Transparency and clarity on the purpose of a task is a main prerequisite for any development done. Readability is essential for any object to ensure future changeability or simply allowing adequate testing of the development. Single-Line/In-Line Comments in every part of the code is essential for better understanding.

Keep it simple!

- Flexibility

Any development should be done in a way that allows for easy future adoption to changed circumstances. Good and detailed documentation is essential for possible modifications or bug- fixes, that might have to be made to the system.

- User-Acceptance

To achieve user acceptance and satisfaction all end-user interfacing development should be as user-friendly as possible.

- Performance

Well performing programs reduce overall runtimes of tasks and are directly related to user acceptance. User-defined programs are one of the main trigger for performance problems. These problems are painful to detect and sometimes even more painful to correct. So please take additional care to avoid them from the beginning.

- General Considerations

1. It is a requirement that no changes be made to standard SAP programs (exception OSS Notes)
2. Requirements should be clearly defined before development is started

(preferably in a signed-off functional specification/Mapping document)

3. Before the creation of a new ABAP, existing programs will be reviewed to determine if they may be used as a template or potentially fill the reporting requirement.
4. All custom development objects have to be named according to the naming standards.
5. Technical Specifications and Unit Test documentation are the responsibility of the developer

- General Programming Rules

The following guidelines must be adhered to, unless approved.

1. All coding must be done within the defined DEV environment.
2. Look to standard functionality first before developing new – standard functionality is supported, modular and decreases the volume of redundant custom code in SAP systems. This significantly reduces Total Cost of Ownership of an SAP system.
3. Existing programs should not be copied and modified – this causes confusion, duplicates code and is difficult to support; instead, the original program should be investigated for modification. If this is not possible a new program may be created.
4. Custom developments will typically be assigned to package related to that module.
5. Internal Tables:
    - Internal table should be defined without header line.
    - Loops at internal tables should be processed by assigning field symbols <fs>. The field symbol then points to the content of the field at runtime. This has a positive effect on performance.
6. Subroutines / Form routines:
    - Use form routines to increase readability of code.
    - All formal parameters have to be typified
7. Standard texts: text elements make the translation of texts possible without changing the source code.
    - The use of text-elements is the appropriate way to output text literals in a report. The use of text elements enables central maintenance of texts and supports translations in other languages.
    - All text elements have to be maintained in English as original language.
    - Set the length of text symbols to be 50% larger than the English

text. Most languages are not as compact as English.

8. BOOLEAN: SAP has no data type 'Boolean' implemented. For all switch elements (e.g. checkboxes, radio buttons etc.) use variables referring to the DDIC-object *BOOLE-BOOLE or XFELD*. The possible values are 'X'= true and SPACE=false.
9. Hard coding:
    - Hard coding of values is not a good practice
    - Use text-elements for printed/screen output values, report headers, etc.
    - Define values as constants in the DATA-part of your program or define within ZTVARVC Table
    - Example: Avoid IF plant = 'US01': the organization may change and there will be added additional plants in the future. Therefore, store the values in ZTVARVC
10. Authority Checks: An effort should be made to find the correct authorization object(s) to be checked. As a rule, authorization checks should be used whenever appropriate to verify the access level of the user executing the program and every report should belong to an authorization group.
11. Table modifications by ABAP programs:
    - ABAP programs that update SAP Standard master and transactional data MUST ALWAYS use SAP transaction codes (where transaction codes are available) by utilizing standard SAP Function Modules, BAPIs, BDC or 'call transaction' utilities. This ensures that logical units of work, rollback, locking operations and edits are performed. SAP tables MUST NEVER be updated directly.
    - ABAP programs MUST NEVER be used to update configuration tables.
12. Error handling:
    - All programs must include proper error handling to avoid undesirable terminations. This means that return codes must be checked after every relevant event in the program.
    - Use the CATCH/ENDCATCH statement to trap runtime errors
    - If there are more than two possible values for the SY-SUBRC field after a performed event, all expected values should be tested explicitly and handled in the program.
    - All Error Logs should be captured and reported to the user.
13. Declaring variables: Use in-line declarations wherever possible
14. Messages – Use of Generic messages (where message just has 4 variables) should be avoided.
15. Wait Statements – These should not be used unconditionally as they can

lead to performance issues.

16. Use Classes wherever possible, since it promotes code reusability and flexibility.

4. Program Documentation

- Header Documentation

A report header must be included in every report .The header documentation contains general information on the program itself and its generation as well as a history on changes made to the code.

The Report Header block should be as follows

```
**********************************************************************
 * Program       : ZXX_EXAMPLE_PROGRAM
 * Author        : Name
 * Created       : MM/DD/YYYY
 * TR#           : XXXXXXXXXX
 * Description   : Example Program which does some sort of reporting
```

- Change Documentation

All changes made to the program after its first release must be tracked and documented. This is done within the Revision Log section of the header and within the code itself. The report change block should be as follows;

```
**********************************************************************
* Changed On :        20/3/2020
*Changed By :              Name
* TR#:                 XXXXXXXXXX
* Defect/Ticket#:      <if applicable>
* Description:
**********************************************************************
```

During code change existing code should NOT be deleted. If no longer relevant to the program it may be removed by commenting out appropriately as shown below.

Any lines added should also be referenced against the reference in the revision block.

Ex:

lv_amount = ls_mseg-dmbtr. "Commented <User ID> <Date> <Defect/ChangeNo>

lv_amount = ls_mseg-wrbtr. "Inserted <User ID> <Date> <Defect/ChangeNo>

Where blocks of code are commented out / inserted these can be highlighted as follows;

*>>>>>> *Start of Comments <User ID> <Date> <Defect/ChangeNo>*
…<Commented Code>
<<<<<<< End of comments < User ID> <Date> <Defect/ChangeNo>

*>>>>>> *Start of Insert <User ID> <Date> <Defect/ChangeNo>*
…<Commented Code>
<<<<<<< End of Insert < User ID> <Date> <Defect/ChangeNo>

5. Performance Guidelines

- ECC Systems (Non -HANA database)

- Use ABAP Inline Declarations (ABAP 7.4)
  https://blogs.sap.com/2017/02/17/why-the-new-open-sql-syntax-is-better/
- Use Primary key or Secondary Index in Database Select WHERE clause
- Do not use SELECT *, mention field names explicitly and in the right order as defined in the table
- SELECT SINGLE not a SELECT … ENDSELECT.
- Use SY-SUBRC = 0 , check after every Table Select
- Avoid SELECT-ENDSELECT – Use Select into table
- Avoid "INTO CORRESPONDING" , define the table fields in the right sequence of selection(defining
  the table will not be needed if New Open SQL is used)

- In most cases Inner Joins are better than FOR ALL ENTRIES, use INNER JOINS wherever possible.
- SELECT statement inside LOOP - Do not write SELECT statements inside a loop, use the FOR ALL ENTRIES
- Before using FOR ALL ENTRIES check that the
  a) Corresponding Internal table is not empty, If the Internal table is empty, the statement
     will select ALL the entries in the Database
  b) The Internal table is sorted by the fields used in the Where Clause, this makes selection faster
- Select Distinct – Do not use Select Distinct, instead select data into the internal table, sort and use
  Delete adjacent duplicates
- No aggregate functions in Select query, all MATH on the application server.
- No Order by /Group by in Select Query
- Use of OR in Where Clause – Even if where clause uses the KEY or Index fields, the optimizer
  stops if the WHERE condition contains an OR expression ,

  Instead of
  SELECT * FROM spfli WHERE carrid = 'LH'
  AND (cityfrom = 'FRANKFURT' OR
  city from = 'NEWYORK')
  Use (both or all key fields in the OR condition)

  SELECT * FROM spfli WHERE (carrid = 'LH' AND cityfrom = 'FRANKFURT')
  OR (carrid = 'LH' AND cityfrom = 'NEWYORK').

- Delete from ITAB –
  Instead of
  LOOP AT <itab> WHERE <field> = '0001'
  DELETE <itab>.
  ENDLOOP.

Use

DELETE <itab> WHERE <field> = '0001'.

- Method calls are efficient than Function Modules, so please use or create Classes wherever possible
- Strictly try to use only field symbols instead of Work Areas, Field symbols are memory pointers and are faster to access.
- When reading a single record in an internal table, the READ TABLE WITH KEY is not a direct READ.
  The table needs to be sorted by the Key fields and the command READ TABLE WITH KEY BINARY SEARCH is to be used
- Moving data from one internal table to the other , Use itab1[ ] = itab2[ ] .
- Sort table by <field1> <field2> , instead of just sort <table> .
- Counting table lines

Instead of

```
LOOP AT <itab>.

counter = counter + 1.

ENDLOOP.
```

Use

```
DESCRIBE TABLE <itab> LINES n.
```

- Avoid Nested Loops – if need be use the parallel cursor technique. Parallel cursor method involves finding out the index using READ statement & searching only the relevant entries.

Instead of Nested loop

```
Loop at T1 into W1.
      Loop at T2 into W2 where F1 = W1-F1.
            Endloop
Endloop
```

Use Parallel cursor method

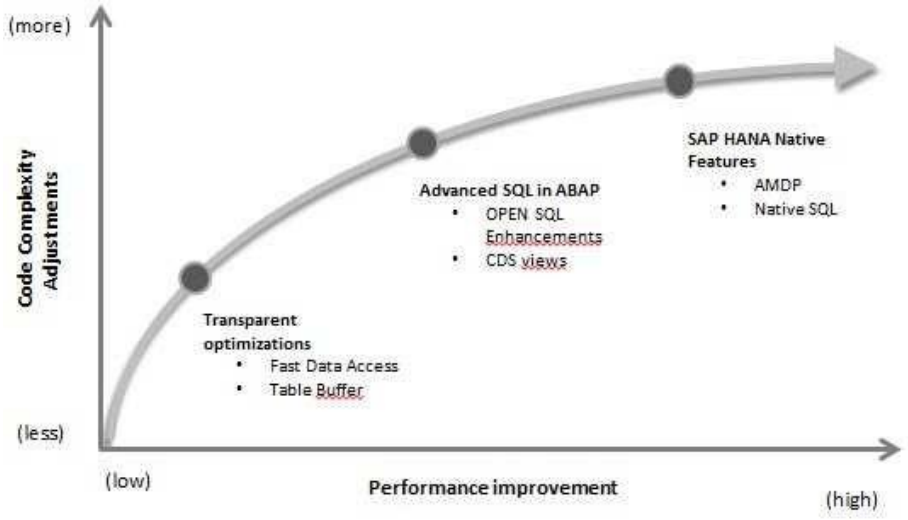
```
Loop at T1 assigning <fs1>.
 Read table T2 assigning <fs> with key F1 = W1-F1 Binary Search.
      l_index = SY-tabix.
            Loop at T2 assigning <fs2> from l_index.
If <fs2>-F1 <> <fs1>-F1.
      Exit.                  " Exit inner loop when condition fails
 Endif.
      Endloop
```

Endloop

- Use CASE statements instead of multiple IF-conditions, as CASE statements are clearer and
  faster to execute
- Free the internal tables/work area/ variables which are no longer used.
- Clear work area at end of every LOOP pass.
- CODE CLEANUP – Remove dead code (Commented code), unused constants, variables, work area and internal tables from the program.
- Avoid LOOP on the same internal table more than once.
- Internal tables with header line not allowed

ECC System with SAP ABAP 7.4SP04 and above – ABAP CDS Views are supported.
So please start using the code push down paradigm. Also, ABAP CDS are database independent, so can be used irrespective of the database.

- S4 HANA Systems – ABAP on HANA .



Transparent optimization	• Improvements in the ABAP Stack (FDA) • Direct benefit without adjustments
New Open SQL - Code Push Down or Code to Data paradigm	• Move all logic and calculations to the Database
CDS (Core Data Services) Views -ABAP CDS	• Database views are used for data access
ABAP Managed Database Procedure (AMDP)	• Stored Procedures

New Open SQL syntax for

CASE Statement

Type 1

```
SELECT f1, f2, f3, f4, f5
CASE <fieldname>
WHEN <condition1>
THEN <Calculation / text /logic>
WHEN <condition2>
THEN <Calculation / text /logic>
ELSE <Default Calculation/text/Logic>
End as <alias1>
FROM <table_name>
INTO TABLE @DATA(<tab1>).          "Inline Declarations
```

Type 2

```
SELECT f1, f2, f3, f4, f5
Case
When <condition2>                  " condition should include fieldnames
THEN <Calculation / text /logic>
ELSE <Default Calculation/text/Logic>          " nested case possible
END AS <Alias_name1 >
FROM <table_name> INTO TABLE @DATA(<tab1>).          "Inline Declarations
```

Aggregate functions with group by and having

```
SELECT a~f1, a~f2, b~f1, b~f2,
COUNT( DISTINCT ( a~f1 ) ) AS <alias1>,
SUM(a~f2) AS <alias3>,
AVG( b~f2 ) AS <alias4>,
```

```
FROM <table1> AS a INNER JOIN <table2> AS b
ON a~f1 = b~f1
INTO TABLE @DATA(<tab1>)
WHERE <condition>
GROUP BY <field_list>
Having <condition> .
```

Other Aggregate and Math functions

Ceil(arg1)	"rounding off to next higher Integer
abs(arg1)	" Absolute of b~f4
floor(arg1)	"rounding off to next lower Integer
round(arg1)	"rounding off to 2 decimal places
min(arg1)	
max(arg1)	

COALESCE

According to SAP documentation, the COALESCE function in Open SQL returns the value of the arg1 (if this is not the null value); otherwise, it returns the value of the arg2.

COALESCE(arg1, arg2)

- Here f1, f2 and f3 are selected if f3 is NOT NULL
  Else f1, f2 and f4 are selected is f3 is NULL .

```
SELECT f1, f2
COALESCE( f3, f4 ) AS <alias>
FROM <table>
INTO TABLE @DATA(<tab>).
```

String expressions

Concatenation using '&&'

```
SELECT f1
&& '(' && f2 && ')' AS <alias>
FROM <table>
INTO TABLE @DATA(<tab>).
```

Others

Concat(arg1,arg2)

Replace(arg1,arg2,arg3)
Substring(arg1,pos, length)
Length(arg1)

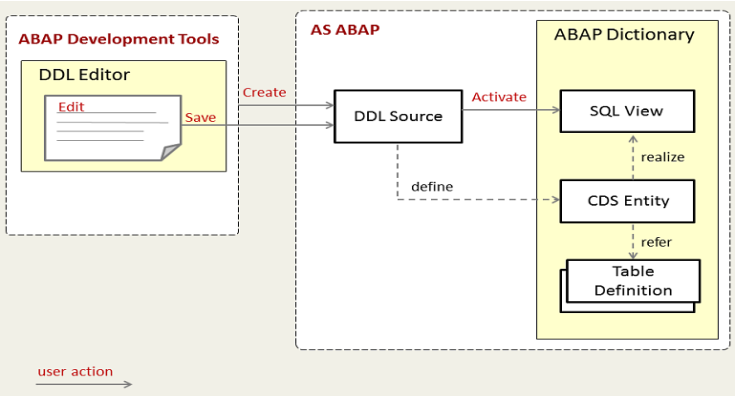
Example :

```
 SELECT b~kschl, b~vkorg, b~vtweg, b~spart , b~kunnr, b~kdgrp, b~matnr, b~matkl,
b~vkbur, b~werks, b~kondm, b~zzmvgr1, b~mfrgr,
      a~knumh AS knumh,
    CASE
       WHEN a~konwa LIKE '#%' ESCAPE '#' THEN division( a~kbetr, 10, 2 )
       ELSE a~kbetr
       END AS kbetr,
       a~konwa AS konwa, a~kmein AS kmein
      FROM konp AS a
      INNER JOIN @lt_atab AS b
      ON a~knumh = b~knumh
      WHERE a~loevm_ko EQ ' '
      INTO TABLE @DATA(lt_konp) .
```

ABAP CDS

Eclipse IDE with ABAP Development Tool (ADT) should be used to create ABAP CDS views

ABAP CDS views only support data read and not write/data modification (DML), i-e only Selects and no updates, inserts or delete on the Table



Name of the DDL Source and CDS Entity should be the same. SQL View name should be different

from the
CDS Entity/View.

Annotations in CDS views add more semantic meaning to the Data Definition/Meta data.

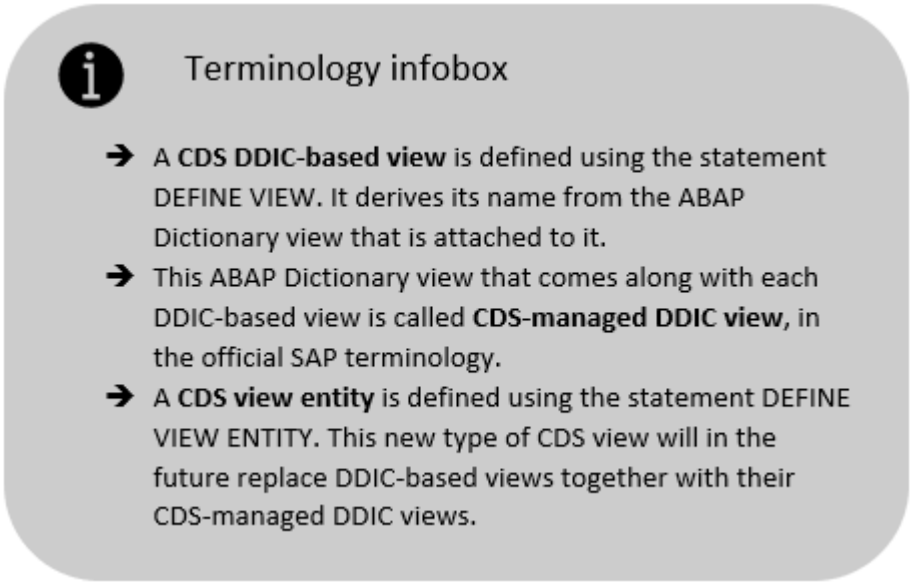
CDS simplifies and harmonizes the way you define and consume your data models, regardless of the consumption technology. Technically, it is an enhancement of SQL which provides you with a
data definition language (DDL) for defining semantically rich database tables/views (CDS entities) and
user-defined types in the database. Some enhancements include:

- Expressions used for calculations and queries in the data model
- Associations on a conceptual level, replacing joins with simple path expressions in queries
- Annotations to enrich the data models with additional (domain specific) metadata

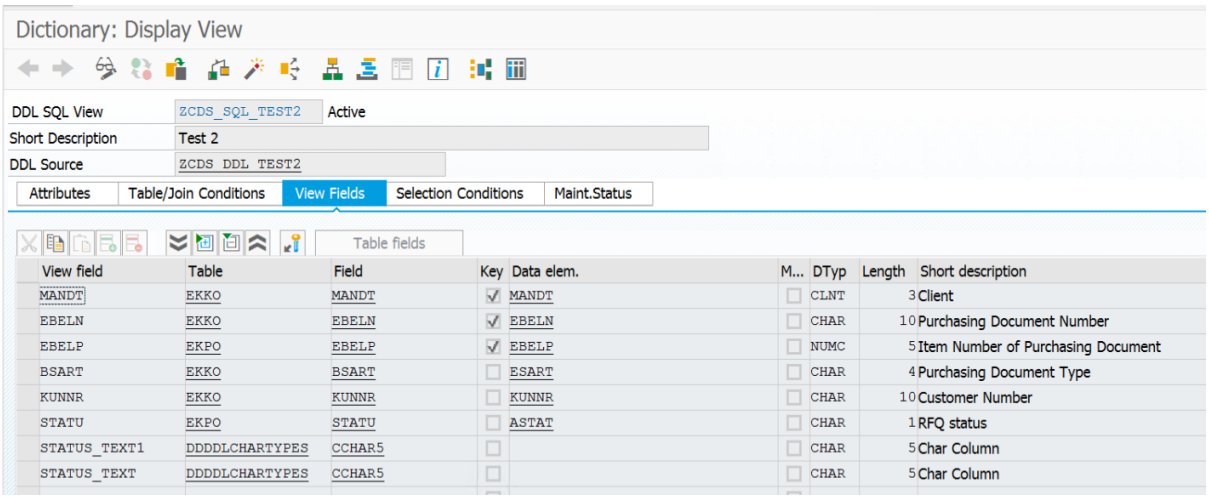
CDS is supported natively in both the ABAP and the HANA Platforms!

Classic DDIC based CDS Views v/s CDS View Entity

With ABAP release 7.55, a new type of CDS view it's called CDS view entity.



DDIC based CDS Views create a SQL/ ABAP dictionary View in the backend upon activation, hence the annotation @AbapCatalog.sqlViewName – is mandatory



In CDS Entities, ABAP dictionary View is not created so the annotation @AbapCatalog.sqlViewName is not required.
Also, further syntaxes are improvised, and some unused features are removed.

https://blogs.sap.com/2020/09/02/a-new-generation-of-cds-views-cds-view-entities/

VDM – Virtual DATA models – Primarily used in Analytics
https://blogs.sap.com/2017/10/09/abap-core-data-services-part-2virtual-data-model-types/

ABAP CDS Views Ready reference includes general ABAP on HANA code push down Syntaxes, AMDP, CDS Table functions ALV IDA's .

ABAP%20CDS.docx

5.3 Analysis Tools

SAP has several tools to help the developer check the efficiency and performance of programs These tools are most useful when run with production like volumes of data. The data in development systems may not be enough to correctly identify the potential bottlenecks or issues.

- Runtime Analysis – Transaction SAT

The runtime analysis tool (transaction SAT) can be used to analyze the performance of specific ABAP code.
http://scn.sap.com/community/abap/testing-and-

troubleshooting/blog/2011/01/18/next-generation- abap-runtime-analysis-sat-how-to-analyze-performance

- ST05 – Performance Trace

The Performance Trace allows you to record database access, locking activities, and remote calls of reports and transactions in a trace file and to display the performance log as a list. The SQL trace element of this tool is particularly useful for analyzing the performance of database accesses of a particular program / transaction.

- Extended Syntax Check

All developments must be checked with extended syntax check (Transaction SLIN) before being releasing the transport. For performance reasons, the normal syntax check does not include many syntactic and semantic checks. Before a program is transported to production, the extended syntax check should display at least no errors.

- Code Inspection

The ABAP Test Cockpit can perform code checks to produce findings (errors and warnings about the code that need to be corrected)

Before releasing the transport is encouraged to, perform a ATC run. This will allow the developer to find and correct problems with his or her code before the Central run does.

5 Code review process

All code should pass through a code review before releasing the Transport.

1) The extended syntax check should display no errors
2) Strictly adhere to all naming conventions and coding conventions mentioned on this document, if there is a Naming convention Document from the Client, it takes priority over this .
3) Code Review Checklist document duly filled

4) Ensure efficient runtime performance
5) Technical Unit Test Document.

6 Appendix A – SAP Module Codes

Module	Code
Financial Accounting – GL/AR/AP/AA	FI
Controlling	CO
Materials Management	MM
Sales and Distribution	SD
Retail	RE
BW/BI	BI